

What is plasmaHub?

Think of it as an **online blood bank**. It's a web application where:

- **Donors** can register and find blood requests
 - **Hospitals** can post blood requirements and manage their stock
 - **Admins** can manage the entire system
-

Project Structure (Simple Analogy)

Imagine a restaurant:

- **Frontend** = The restaurant's **menu and ordering counter** (what customers see)
- **Backend** = The **kitchen** (where orders are processed)
- **Database** = The **storage room** (where ingredients are kept)

Code

plasmaHub/

├── frontend/ ← What users see (React)
├── backend/ ← The actual logic & processing (Node.js)
└── package.json ← Configuration files

Frontend (React) - "What Users See"

What it does:

- Displays beautiful web pages and forms
- Takes user input (login, blood requests, etc.)
- Sends data to the backend

Key Pages:

1. **Login/Signup** - User authentication
2. **Dashboard Pages** - Different views for different users:
 - Admin sees: users list, hospitals list, all requests
 - Donor sees: available blood requests, their profile
 - Hospital sees: blood stock, incoming requests

Technology: React, Bootstrap (for styling), Google Charts (for graphs)

Backend (Node.js + Express) - "The Kitchen"

What it does:

- Receives requests from the frontend
- Processes the data
- Stores/retrieves data from the database
- Sends responses back

Main Components:

1. Server (server.js)

Code

Think of it as the restaurant's main counter.

It listens for orders (HTTP requests) and directs them.

2. Routes (router/)

Code

Like different counters in a restaurant:

- Counter 1: "Blood Requests"
- Counter 2: "User Management"
- Counter 3: "Blood Stock"

Each handles specific tasks.

3. Controllers (controller/)

Code

Like the chefs in the kitchen.

When you order something, the chef does the actual work.

Examples:

- authController → Handles login/signup
- userController → Manages user data
- bloodStockController → Updates blood inventory
- requestController → Processes blood requests

4. Models (model/)

Code

Like the recipe cards in a kitchen.

They define the structure of data:

- User recipe: name, email, blood type, etc.
- Blood stock recipe: blood type, quantity, expiry date, etc.
- Request recipe: donor name, hospital, urgency, etc.

5. Database (MongoDB)

Code

Like the storage room of the restaurant.

Everything is stored here:

- User information
 - Blood inventory
 - Request history
-

How It All Works Together (Simple Flow)

Scenario: A hospital wants to request blood

1. Frontend (React):

- User fills a form: "I need O+ blood, quantity: 5 units"
- Clicks "Submit"

2. Network (Axios):

- Sends this data to the backend

3. Backend (Express):

- Request arrives at the router
- Router says: "This is a blood request, send to requestController"

4. Controller:

- Takes the request data
- Validates it (is it correct format?)
- Creates a new request in the database

5. Model & Database:

- Data is saved in MongoDB following the request structure

6. Response:

- Backend sends back: "Success! Request saved"

7. Frontend:

- Shows: "Your blood request has been posted!"

Key Technologies Explained

Part	Technology	Why?
Frontend	React	Makes interactive web pages
Backend	Node.js + Express	Fast, easy to build APIs
Database	MongoDB	Stores all data (flexible)
Styling	Bootstrap	Makes it look nice
Email	Nodemailer	Sends notifications

What You Need to Know as a Fresher

1. Frontend & Backend are separate:

- Frontend asks questions (HTTP requests)
- Backend answers them (HTTP responses)

2. Each part has a specific job:

- Frontend = Pretty UI
- Backend = Logic & Processing
- Database = Storage

3. Three types of users:

- **Donor** → Can offer blood
- **Hospital** → Can request blood
- **Admin** → Manages everything

4. **MVC Pattern** (you'll hear this a lot):
- **Model** = Data structure (what data looks like)
 - **View** = Frontend (what user sees)
 - **Controller** = Logic (what happens when user clicks)
-

Common Interview Questions You Might Get

Q: Why separate frontend and backend?

- A: So one person can work on design while another works on logic. Also, the same backend can work with mobile app, web, etc.

Q: What's a model?

- A: It defines the structure of data. Like a blueprint saying "a User must have name, email, password."

Q: How does data flow?

- A: User → Frontend → Backend → Database (and reverse for responses)

Q: Why MongoDB?

- A: It's flexible and easy to store different types of data without strict structure.
-

This is basically a **full-stack application** where you have a complete system handling blood donations! 📌